

7. Registros

Un registro es un bloque formado por biestables cuya función es almacenar varios bits de información, en un principio tantos como biestables formen el registro.

La estructura interna de los registros y los biestables que contienen puede ser muy variada. El objetivo de este tema no será profundizar en todas las variantes posibles, sino centrarnos en comprender su funcionamiento y estructuras más habituales.

7.1. Los registros y sus características

La primera clasificación de registros la podemos ver desde el punto de vista de cómo se introduce la información que se quiere almacenar y cómo se recupera dicha información. En este sentido existen registros que introducen los bits de información de forma secuencial, es decir, en serie, y registros que introducen todos los bits de información simultáneamente, es decir, en paralelo. De la misma manera podemos tener registros en los que la información se recupera en serie, es decir, bit a bit, o bien registros en los que la información se recupera en paralelo, es decir, todos los bits a la vez. Por consiguiente y según el proceso que se sigue para introducir y extraer la información tenemos cuatro posibles modelos de registros: entrada paralelo – salida paralelo (EP/SP), entrada serie – salida serie (ES/SS), entrada serie – salida paralelo (ES/SP) y entrada paralelo – salida serie (EP/SS).

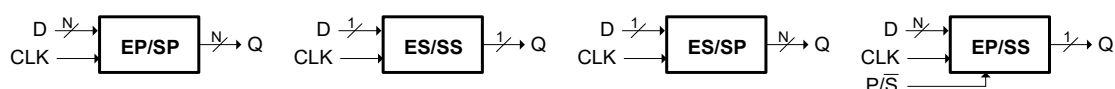


Figura 7.1. Clasificación de los registros según si el proceso que se sigue para introducir y extraer la información se realiza en serie o en paralelo. Vemos que aparte de las líneas de entrada y salida de información, indicadas por D y Q respectivamente, todos los registros presentan una señal de reloj que marcará (sincronizará) el momento en el que la información ha de ser introducida o bien extraída. Observamos que los registros EP/SS disponen de una entrada externa que los otros no presentan y que explicaremos más adelante llegado el momento.

La introducción o extracción de información en paralelo presenta como ventaja el hecho que tan solo es necesaria una única activación de la señal de reloj para que todos los bits sean introducidos o extraídos. En este sentido será más rápido que la introducción o extracción de información en serie, puesto que ésta última precisará de varias activaciones de la señal de reloj, también denominadas ciclos de reloj, para finalizar el proceso en tanto que a cada ciclo de reloj sólo se introduce o extrae un único bit.

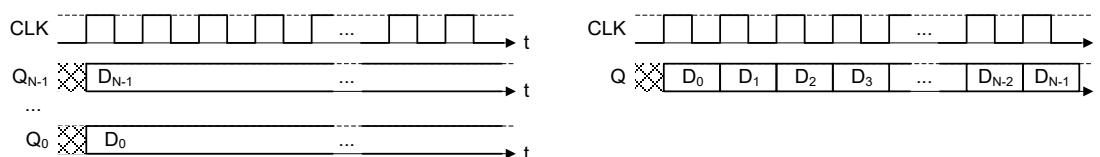


Figura 7.2. Comparación entre la extracción de la información cuando se transmite en paralelo respecto a cuando se transmite en serie. Podemos ver que en el primer caso (evolución de la

izquierda) cuando se activa la señal de reloj todos los bits de salida son actualizados. En el segundo caso (evolución de la derecha) a cada ciclo de reloj sólo se actualiza uno de los bits de información y por consiguiente es necesario activar tantos ciclos de reloj como bits queramos extraer.

Por otro lado, la introducción o extracción de información en paralelo requiere tener un dispositivo con múltiples entradas y/o salidas, es decir, con múltiples *pins* que se tendrán que conectar a otros dispositivos y por consiguiente el cableado entre ellos será más complejo. En cambio, la introducción o extracción de información en serie requiere de una única línea de entrada y/o salida, es decir de un único *pin*, facilitando de esta manera el cableado con otros dispositivos.

Aparte de la señal de reloj y de las líneas de entrada y salida de información, los registros suelen presentar otras señales de control. Las más comunes son las siguientes:

- *Reset* (o bien *Clear*). Es una señal de entrada que cuando se activa pone a cero todos los biestables del registro encargados de almacenar la información.

La activación puede ser de forma síncrona o bien de forma asíncrona. En el primer caso los biestables se ponen a cero cuando la señal de reloj pasa de cero a uno (flanco de subida) si en ese instante está activada la entrada de *Reset*. En el segundo caso se ponen a cero justo en el mismo instante en el que se active la señal de *Reset*.



Figura 7.3. Comparación entre la puesta a cero de los valores de un registro de manera síncrona (evolución de la izquierda) y de manera asíncrona (evolución de la derecha).

Esta señal puede ser activa por lógica positiva, es decir cuando adopta el valor lógico 1, o bien por lógica negativa, es decir cuando adopta el valor lógico 0. En este último caso a menudo se identifica por $\overline{Reset}$ o $\overline{Clear}$.

- *Set* (o bien *Preset*). Es una señal de entrada que cuando se activa pone a uno todos los biestables del registro encargados de almacenar la información. Nuevamente el funcionamiento puede ser de tipo síncrono o bien asíncrono.

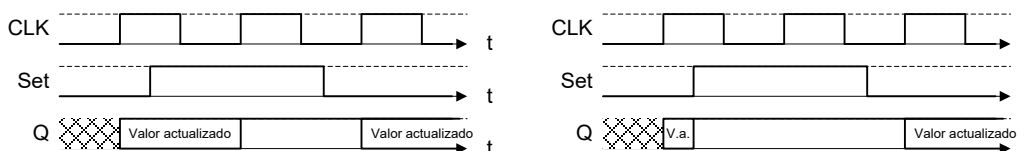


Figura 7.4. Comparación entre la puesta a uno de los valores de un registro de manera síncrona (evolución de la izquierda) y de manera asíncrona (evolución de la derecha).

Como en el caso anterior esta señal puede ser activa por lógica positiva o bien por lógica negativa y en este último caso a menudo se identifica por $\overline{Set}$ o $\overline{Preset}$.

Comentar que si bien es habitual encontrar registros con entradas tipo *Reset* no es tan frecuente hallar registros con entradas tipo *Set*.

- *Enable* (o bien *G*). Es una señal de control que habilita a los biestables del registro encargados de almacenar la información a actualizar sus valores cuando la señal de reloj pasa de cero a uno (flanco de subida). En caso de no estar habilitados, por mucho que llegue una señal de reloj, los biestables que configuran el registro no actualizarán su valor.

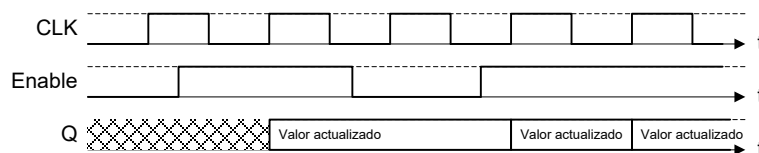


Figura 7.5. Señal de *Enable*. Vemos que el biestable actualizará su salida a cada flanco de subida de la señal de reloj siempre y cuando la señal de *Enable* esté activa. En caso contrario el biestable mantiene el valor que tuviera anteriormente.

En este caso no tiene sentido hablar de *Enable* de manera síncrona o asíncrona, pero sí que tiene sentido hablar de *Enable* que es activo por lógica positiva, es decir que habilita el registro cuando está a uno, o bien por lógica negativa, es decir que habilita el registro cuando está a cero. En este último caso se suele denominar *Enable*.

- *Output Enable* (o bien *OE*). Esta señal habilita que por los *pins* de salida del registro se muestre el valor de salida de los biestables contenidos en el registro o bien, en caso de no estar activada, que la salida del registro quede en alta impedancia, es decir como un circuito abierto.

Esta función se consigue mediante un elemento denominado *tri-state*. Tal y como se intuye por su nombre, un *tri-state* es un elemento que puede presentar tres estados: el cero lógico, el uno lógico o bien alta impedancia (a menudo designado por una *Z*). Presenta siempre una entrada de control que cuando está activa hará que su salida adopte el mismo valor que su entrada, pudiendo ser un cero lógico o bien un uno lógico. En cambio, cuando la señal de control del *tri-state* está desactivada entonces la salida adopta el valor de alta impedancia. Esta señal de control puede funcionar tanto por lógica positiva como por lógica negativa, según el modelo de *tri-state*.

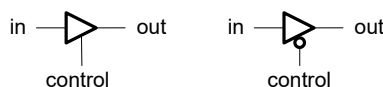


Figura 7.6. Símbolo de un elemento *tri-state*. Cuando la señal de control está habilitada, por uno en la figura de la izquierda y por cero en la figura de la derecha, entonces *out* adopta el mismo valor que *in*, es decir $out = in$. En caso contrario la salida *out* queda en alta impedancia, como un circuito abierto, es decir $out = Z$.

Los registros suelen disponer de elementos *tri-state* justo antes de las salidas físicas del dispositivo, es decir, justo antes de los *pins* de salida. De esta manera en función de la señal *Output Enable* habilitan que los *pins* de salida muestren los valores contenidos en los biestables del registro o bien que queden en alta impedancia. Se suelen utilizar para conectar conjuntamente las distintas salidas de un registro a un mismo elemento y para evitar colisiones entre ellas se hace que sólo uno de los registros tenga sus salidas habilitadas.

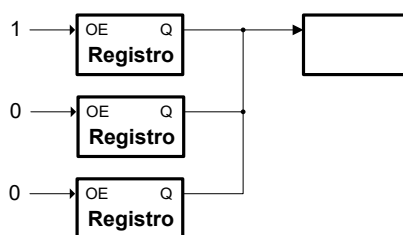


Figura 7.7. Uso de la señal de Output Enable. Vemos que podemos conectar entre sí las salidas de varios registros que han de enviar una cierta información a otro dispositivo sin que exista colisiones entre los datos de cada uno de ellos en tanto que sólo uno de los registros tiene su salida habilitada y por consiguiente los otros están en alta impedancia, es decir en circuito abierto.

En esta sección pretendemos que tengáis una visión general de los distintos tipos de registros, así como de las señales de control que pueden presentar. Más adelante, a medida que vayamos comprendiendo el funcionamiento y la estructura básica de los cuatro tipos fundamentales de registros (EP/SP, ES/SS, ES/SP, EP/SS) entonces iremos recuperando los conceptos de estas señales de control para ver cómo se implementan internamente.

7.2. Registros EP/SP

Como hemos comentado anteriormente, los registros de entrada paralelo y salida paralelo (EP/SP) almacenan toda la información en un único flanco de reloj.

Un registro EP/SP de N bits está compuesto internamente por N biestables, consideraremos de tipo D, cada uno de los cuales almacenará uno de los N bits de entrada. Su estructura interna es por consiguiente muy simple en tanto que se limita a disponer N biestables en paralelo, cada uno de los cuales está conectado a una de las líneas de entrada y todos ellos actualizan su salida según la señal de reloj.

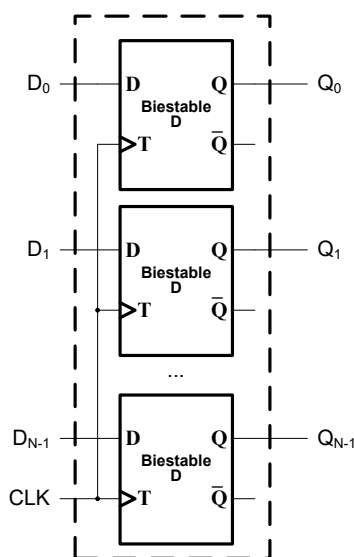


Figura 7.8. Estructura básica de un registro EP/SP. Vemos que presenta tantos biestables como bits almacena el registro. Cada biestable está conectado a una de las entradas del registro y todos ellos comparten el sincronismo. De esta manera, en cada flanco de subida del sincronismo CLK los biestables actualizan sus salidas según los valores que contengan las entradas del registro.

Ejemplo 7.1

Para comprender el funcionamiento de un registro EP/SP veamos la evolución temporal de un registro EP/SP de 4 bits como el siguiente, sin más entradas de control que la señal de reloj.



Figura 7.9. Registro EP/SP de 4 bits sin entradas de control más que la señal de reloj CLK .

Para ello supondremos que las entradas $D[3..0]$ varían según un cierto cronograma (gráfico que muestra la evolución temporal de señales) y analizaremos el por qué las salidas adoptan los correspondientes valores.

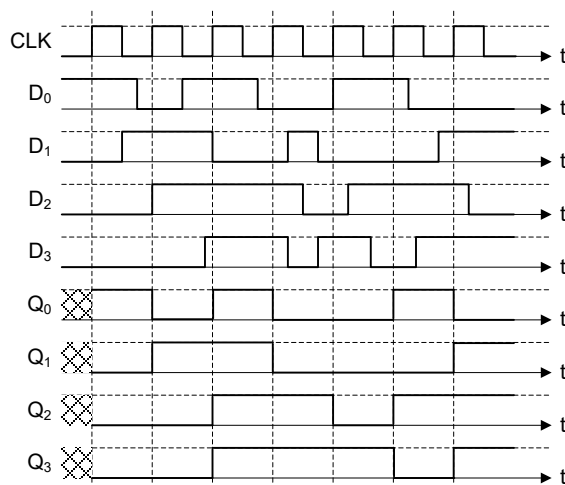


Figura 7.10. Cronograma que muestra la evolución a lo largo del tiempo de un registro EP/SP de 4 bits para una supuesta evolución de las señales de entrada $D[3..0]$. Para facilitar la lectura del gráfico hemos indicado con unas líneas discontinuas verticales los instantes en los que la señal de reloj pasa de cero a uno (flanco de subida), al corresponderse con los instantes en los que los biestables internos del registro actualizan sus salidas Q según los valores que en ese momento presenten sus entradas D . Así por ejemplo en el primer flanco la entrada D_0 vale uno y por ese motivo en ese instante Q_0 adopta el mismo valor. En el siguiente flanco D_0 vale cero y por ese motivo Q_0 adopta este nuevo valor. Así sucesivamente podemos ir mirando en cada flanco de subida de CLK el valor que presenta una cierta entrada D y ver como la correspondiente salida adopta el mismo valor.

Es importante mencionar qué ocurre en el supuesto que la señal de reloj pase de cero a uno (flanco de subida) justo en el mismo momento en el que la entrada del biestable cambia de valor. En este caso consideraremos que la salida del biestable adopta el valor que presentaba antes de dicho flanco, puesto que en la realidad todos estos bloques presentan un cierto retardo a la hora de actualizar sus valores y por consiguiente cuando se produce un flanco de subida del reloj el biestable no tiene tiempo de “leer” el nuevo valor. Vemos varios de estos casos en el ejemplo anterior; por ejemplo, el quinto flanco de subida para la entrada D_0 o bien el tercer flanco de subida para la entrada D_1 .

Ejemplo 7.2

Para practicar la evolución de las señales de salida de los biestables ante una cierta evolución temporal de las entradas, os facilitamos unas supuestas entradas D para que miréis de dibujar las correspondientes salidas Q .

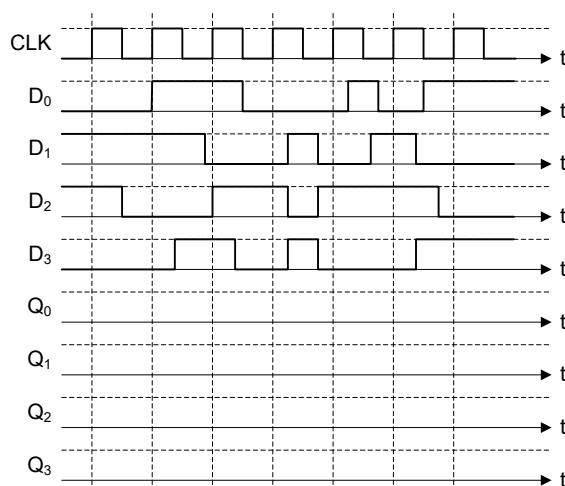


Figura 7.11. Cronograma que muestra la evolución a lo largo del tiempo de un registro EP/SP de 4 bits para una supuesta evolución de las señales de entrada $D[3..0]$ con el propósito que deduzcáis la evolución de las correspondientes salidas.

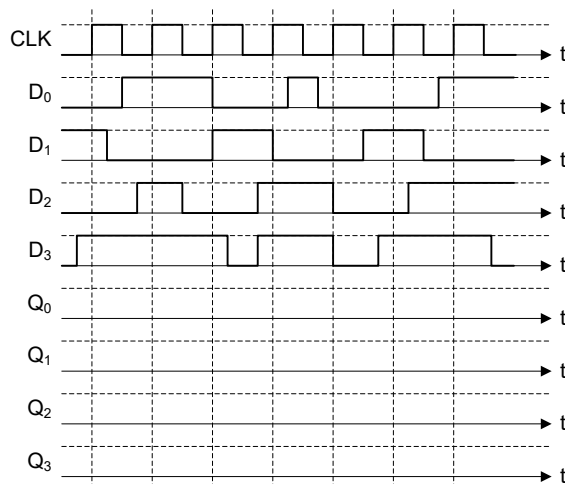


Figura 7.12. Cronograma que muestra la evolución a lo largo del tiempo de un registro EP/SP de 4 bits para una nueva supuesta evolución de las señales de entrada $D[3..0]$ con la finalidad que deduzcáis la evolución de las correspondientes salidas.

7.3. Registros con desplazamiento serie: ES/SS, ES/SP y EP/SS

Existen registros en los que la información se almacena en serie, es decir, en lugar de tener múltiples líneas de entrada tan solo poseen una única línea por la que se introduce la información, bit a bit, a medida que se aplican flancos de subida en la señal de reloj. De la misma manera existen registros en los que la información se extrae en serie y por

consiguiente sólo precisan de una única línea de salida por la que se extrae la información a cada flanco de la señal de reloj.

De esta manera podemos tener registros entrada serie – salida serie (ES/ES), entrada serie – salida paralelo (ES/SP) y entrada paralelo – salida serie (EP/SS). En todos ellos se realiza una transmisión en serie de la información.

7.3.1. Registros ES/SS

La arquitectura básica de los registros ES/SS de N bits está constituida por una cadena de N biestables en los que la salida de cada uno de ellos se conecta a la entrada del siguiente, compartiendo todos ellos la misma señal de sincronismo. De esta manera en cada flanco del sincronismo tendremos que cada biestable almacena la información del biestable anterior, propagando de esta manera la información en serie, desde la entrada del primer biestable hasta la salida del último.

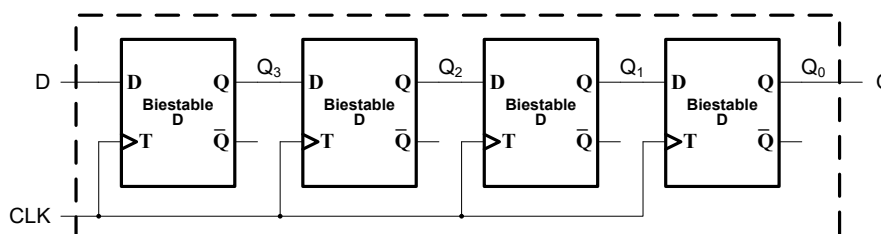


Figura 7.13. Esquema de un registro ES/SS de 4 bits. Podemos ver que cuando se produce un flanco de reloj el primer biestable (de la izquierda) almacena en Q_3 el valor de la entrada D . Simultáneamente el segundo biestable almacena en Q_2 el valor que tenía el anterior biestable en Q_3 antes que éste almacenara el valor de D . Simultáneamente ocurrirá lo mismo con el tercer biestable que en Q_1 almacenará el valor que había en Q_2 y con el cuarto biestable que almacenará en Q_0 el valor que había en Q_1 . Podemos observar cómo la información se propaga en serie con cada flanco de subida del reloj externo.

Este tipo de estructuras permite minimizar el número de líneas requeridas para introducir y para extraer la información. No obstante el almacenamiento o recuperación de la información es más lento que en paralelo puesto que precisaremos $N-1$ ciclos de reloj para que la información que entra en el registro se desplace hasta su salida.

Ejemplo 7.3

Para practicar los cronogramas de los registros ES/SS veamos, respecto el registro ES/SS de 4 bits que acabamos de ver, la evolución de los bits a lo largo de todos los biestables si la línea de entrada D evoluciona de la forma indicada en el siguiente cronograma.

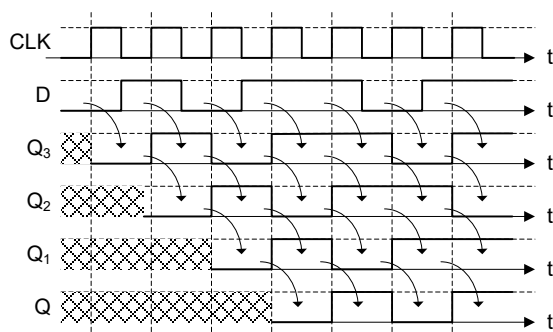


Figura 7.14. Ejemplo de cronograma para un registro ES/SS sobre el que hemos marcado con unas flechas cómo se propaga la información desde un biestable hacia el siguiente. Estas flechas normalmente no se trazan en los cronogramas y sólo se han indicado en este ejemplo para facilitar la lectura del mismo. Así estamos viendo que por el registro hemos ido introduciendo la secuencia 0101101... y esta información se ha ido propagando en serie hasta que por Q está saliendo la misma información 0101... a partir del tercer ciclo de reloj.

Ejemplo 7.4

Para practicar los cronogramas de los registros ES/SS os proponemos que, respecto al registro ES/SS de 4 bits que acabamos, de ver dibujéis la evolución de los bits a lo largo de todos los biestables si la línea de entrada D evoluciona de las siguientes formas.

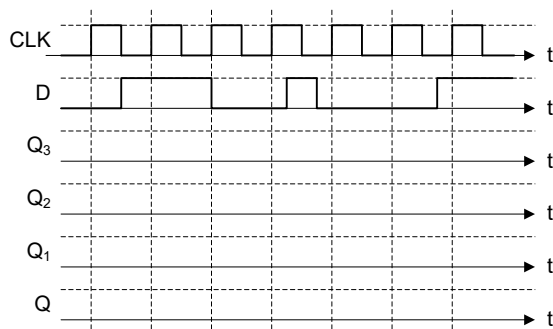


Figura 7.15. Cronograma para un registro ES/SS de 4 bits sobre el que trazar la evolución de los bits a lo largo de los distintos biestables.

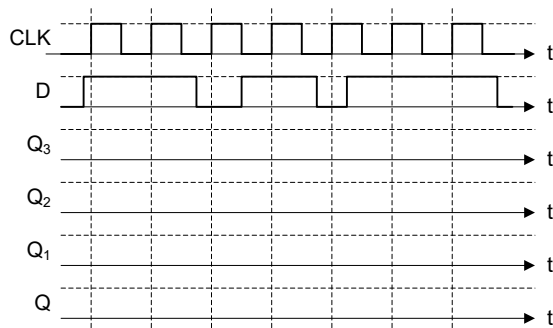


Figura 7.16. Cronograma para un registro ES/SS de 4 bits sobre el que trazar la evolución de los bits a lo largo de los distintos biestables.

7.3.2. Registros ES/SP

Los registros ES/SP son aquellos que presentan la misma arquitectura interna que los registros ES/SS pero que extraen la información de salida de cada uno de los biestables hacia el exterior del registro. Es decir, presentan una única línea por la que introducen la información en serie, pero tienen N *pins* de salida por los que se puede visualizar la información contenida en cada uno de los biestables que configuran el registro.

Sus cronogramas serán pues los mismos que acabamos de ver. La única diferencia radica pues en el hecho que mientras que antes, por mucho que los dibujamos, desde el punto de vista de usuario del registro sólo podríamos acceder a la información de Q_0 , ahora podemos acceder, en cualquier momento, a la información de todos los biestables del registro.

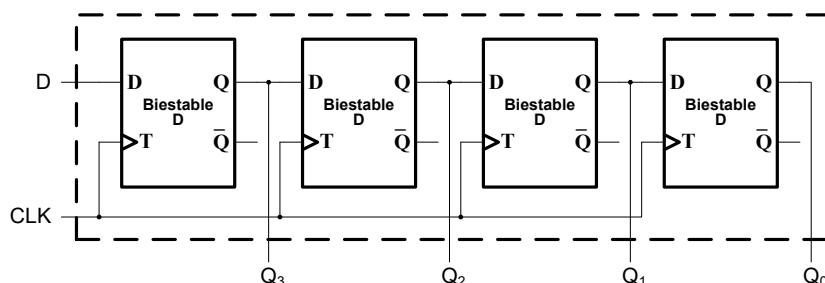


Figura 7.17. Esquema de un registro ES/SP de 4 bits. Podemos ver que es idéntico al ES/SS salvo que todas las líneas de salida de los biestables se pueden visualizar desde el exterior al disponer de N *pins* de salida.

7.3.3. Registros EP/SS

En el caso de los registros EP/SS la arquitectura interna sí que varía respecto a los dos casos anteriores. Esto es debido al hecho que la información ciertamente se transmite en serie, de un biestable hacia el siguiente, pero en cualquier momento podemos cargar una cierta información hacia todos los biestables desde el exterior.

Para ello precisa de una señal externa de control que no existía en los anteriores registros, la cual indica si se ha de realizar una transmisión en serie de la información por el interior del registro o bien se ha de efectuar una carga en paralelo de la información proveniente del exterior, en ambos casos de manera síncrona cuando se efectúe un flanco de subida de la señal de reloj. Supondremos que esta señal externa se denomina $P/\bar{S}$ la cual indica que cuando vale 1 efectuamos una carga en paralelo mientras que cuando vale 0 efectuamos una transmisión en serie. Para ello tendremos que diseñar un circuito específico a la entrada de cada biestable Q_i que seleccione la información a introducir llegado un flanco de reloj, pudiendo ser ésta la información de salida del biestable anterior Q_{i-1} o bien la de una línea que proviene del exterior D_i . Existen múltiples opciones para su implementación y podemos ver como ejemplos soluciones con multiplexores y soluciones con elementos *tri-state*.

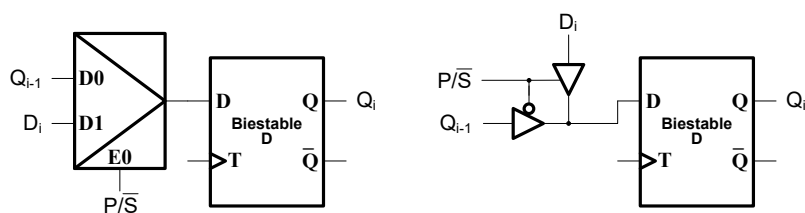


Figura 7.18. Posibles implementaciones de una solución que permita introducir los datos a un biestable a partir de dos posibles fuentes de información que serán, o bien el biestable anterior Q_{i-1} o bien una línea exterior D_i . En el primer caso (de la izquierda) la señal $P/\bar{S}$ aplicada al multiplexor elige entre una u otra opción. En el segundo caso esta señal habilita un *tri-state* o el otro a fin que la información del *tri-state* habilitado llegue a la entrada del biestable. En cualquier caso es importante observar que hasta que no llegue una señal de reloj el biestable no almacenará la información que esté en ese momento en su entrada.

De esta manera para implementar un registro EP/SS lo que tendremos que hacer es replicar cualquiera de las soluciones que acabamos de ver en los múltiples biestables que contenga el registro en cuestión. La única excepción será el primero de los biestables que sólo puede tomar la información del exterior.

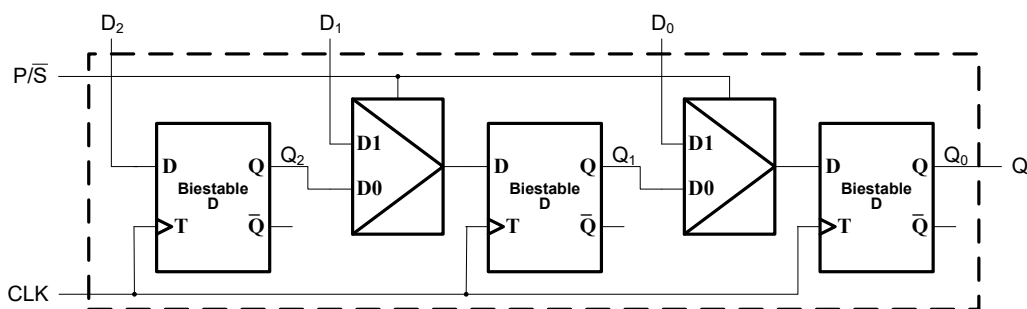


Figura 7.19. Arquitectura interna de un registro EP/SS de 3 bits. Vemos que según el valor de la señal de control $P/\bar{S}$ elegiremos para cada biestable la información de salida del biestable anterior o bien de una línea externa, siendo la única excepción el primer biestable puesto que éste sólo puede tomar la información del exterior.

Ejemplo 7.5

Podemos ver un ejemplo de cronograma para un registro EP/SS de 3 bits.

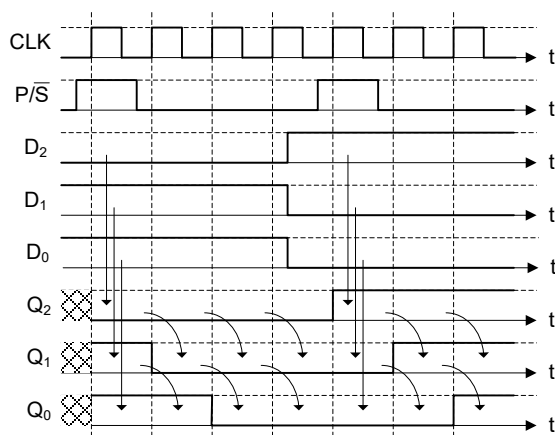


Figura 7.20. Ejemplo de cronograma para un registro EP/SS de 3 bits donde hemos marcado con flechas de dónde proviene la actualización de cada biestable en el momento de producirse un flanco de subida del reloj. Podemos ver que existen un par de ciclos en los que efectuamos una carga en paralelo de la información exterior de las líneas D_2 , D_1 y D_0 . En el resto de ciclos la información de cada biestable proviene de la salida del biestable anterior.

Ejemplo 7.6

Para practicar el funcionamiento de estos registros EP/SS podemos facilitar los siguientes cronogramas en los que sólo se muestra la evolución de las señales de entrada para que miréis de deducir cómo se actualizan los biestables en cada ciclo de reloj. Supondremos un biestable EP/SS de 3 bits.

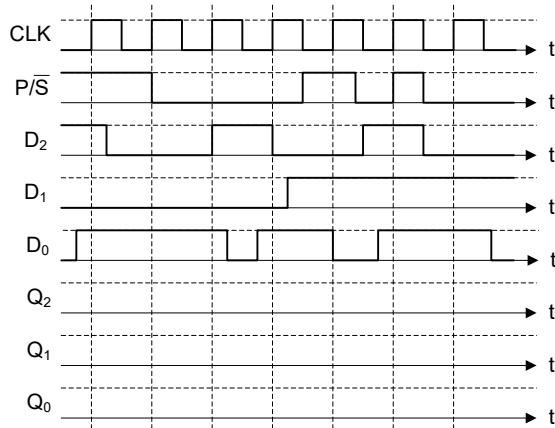


Figura 7.21. Ejemplo de cronograma para un registro EP/SS de 3 bits para que miréis de deducir la evolución de las líneas de salida de cada uno de los biestables internos.

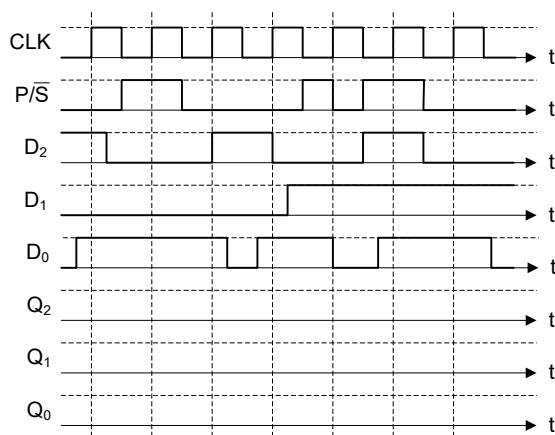


Figura 7.22. Ejemplo de cronograma para un registro EP/SS de 3 bits para que miréis de deducir la evolución de las líneas de salida de cada uno de los biestables internos una vez hemos variado la evolución de la señal de control $P/\bar{S}$.

7.4. Registros comerciales

En este punto ya conocemos cuáles son las estructuras más comunes en registros. En el mercado podemos hallar circuitos integrados que facilitan estas soluciones, los más habituales de los cuales son los que se muestran en la siguiente tabla.

Dispositivo	Descripción
74173	EP/SP de 4 bits con <i>Output Enable</i>
74175	EP/SP de 4 bits
74273	EP/SP de 8 bits
74374	EP/SP de 8 bits con <i>Output Enable</i>
7491	ES/SS de 8 bits
74164	ES/SP de 8 bits
74165	EP/SS de 8 bits
74179	Universal de 4 bits

Tabla 7.1. Circuitos integrados más habituales de la serie 74 que implementan registros.

Algunos de ellos vemos que presentan señales de control adicionales, como pueden ser señales de *Output Enable*. Por otro lado también vemos que existen registros de tipo universal que pueden configurarse para que trabajen en distintos modos. Dejamos para vosotros que miréis por Internet algún catálogo de estos registros y curioseéis su funcionamiento.

7.5. Diseño de las señales de control en registros

La estructura básica de un registro de N bits está formada por N biestables que comparten la señal externa de sincronismo. Según si es un registro de tipo EP/SP o bien de desplazamiento serie (ES/SS, ES/SP, EP/SS) estos registros presentan sus entradas

directamente hacia el exterior (EP/SP) o bien cada biestable hacia su biestable anterior cuando se efectúa un desplazamiento serie.

A partir de estas estructuras básicas que ya hemos visto en los apartados anteriores, podemos incluir señales de control que permitan realizar algunas funciones adicionales: señales de *Enable* que habilitan o no que el registro actualice su valor según el dato que tenga en su entrada cuando exista un flanco de reloj, señales de *Output Enable* que habilitan que las salidas muestren los valores almacenados o bien estén en alta impedancia, señales de *Reset* que ponen los biestables a cero o señales de *Set* que ponen los biestables a uno. Además algunas de estas señales de control, concretamente las de *Reset* i *Set*, pueden funcionar de manera síncrona, es decir, cuando hay un flanco de reloj, o bien asíncrona, en el mismo momento en el que se activan. Veamos a continuación las distintas variantes que podemos tener.

7.5.1. Implementación de la función *Enable*

La señal de *Enable* habilita que llegado un flanco de sincronismo los biestables que configuran el registro actualicen su valor con el dato que disponen en su entrada o bien mantengan el valor que ya tenían. Habitualmente se implementa mediante un multiplexor ubicado en la entrada del biestable que elija una u otra opción.

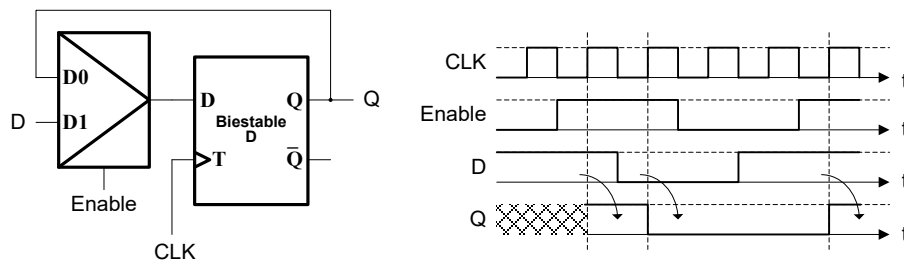


Figura 7.23. Estructura básica que implementa la función de *Enable*. Vemos que, si la señal está habilitada, cuando llegue un sincronismo el biestable se actualizará con un nuevo valor, y si no, con el mismo que ya tenía. El nuevo valor puede provenir del exterior o bien de un biestable anterior, según el tipo de registro.

Aparentemente esta función se podría implementar mediante una puerta lógica de tipo AND a la entrada del sincronismo del biestable a la que se le aplica tanto la señal de *Enable* como la señal de reloj. No obstante, esta solución puede comportarse erróneamente puesto que el biestable sólo ha de actualizar su valor cuando exista un flanco de subida de la señal de reloj y esta solución puede dar lugar a otras situaciones que den lugar a una actualización.

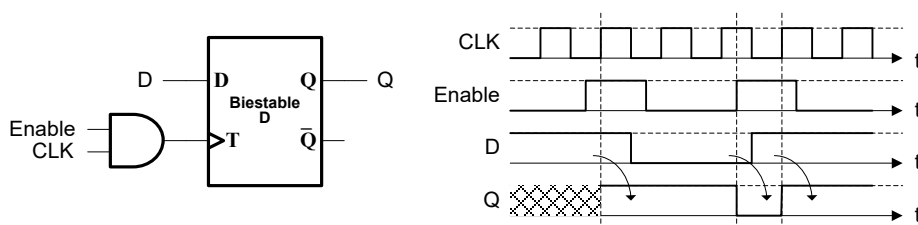


Figura 7.24. Estructura errónea para implementar la función de *Enable*. Vemos en el cronograma que si bien la primera y tercera actualización del biestable son correctas puesto se producen en un flanco de subida del reloj estando la señal de *Enable* habilitada, en el caso de la segunda actualización ésta se produce en un momento en el que no se produce ningún flanco de subida de la señal de reloj.

7.5.2. Implementación de la función *Output Enable*

La señal de *Output Enable* (*OE*) habilita que en los pins de salida del registro se pueda obtener el valor de salida de los biestables o bien que queden en alta impedancia, es decir, como un circuito abierto. Esto se consigue directamente y simplemente poniendo un dispositivo *tri-state* justo antes de la salida del registro.

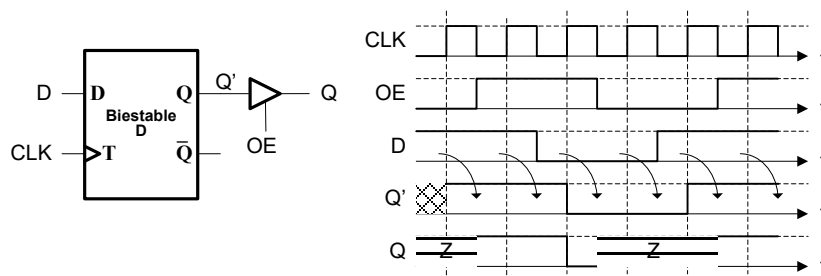


Figura 7.25. Estructura para implementar la función de *Output Enable*. Vemos en el cronograma que si bien la salida del biestable Q' se actualiza con los flancos de subida de la señal de reloj, la salida física real Q sólo muestra el valor de Q' cuando *Output Enable* está habilitado. Para el resto de casos la salida Q está en alta impedancia (Z), es decir, en circuito abierto.

7.5.3. Implementación de la función *Reset*

La señal externa de *Reset* se usa para poner a cero los biestables. Esto se puede realizar tanto de forma síncrona como asíncrona.

En el primer caso el biestable se pone a cero en un flanco de subida del reloj estando *Reset* activo. Se realiza forzando que en la entrada del biestable aparezca un cero cuando se activa el *Reset* con la finalidad que llegado el sincronismo actualice a cero el biestable. Para conseguirlo se pone un multiplexor a la entrada del biestable que selecciona directamente un cero.

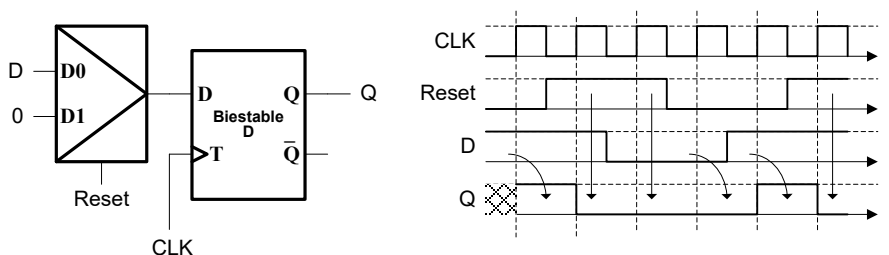


Figura 7.26. Estructura para implementar la función de *Reset* de manera síncrona. Vemos que cuando esta señal está activada la salida se actualiza a cero cuando llega un flanco de subida del reloj mientras que en caso contrario se actualiza según el valor de la entrada D .

En el segundo caso, en el *Reset* asíncrono, el biestable se pone a cero en el mismo momento en el que se activa el *Reset*, esté como esté el sincronismo. Esto se consigue actuando directamente sobre la señal asíncrona de *Reset* del biestable.

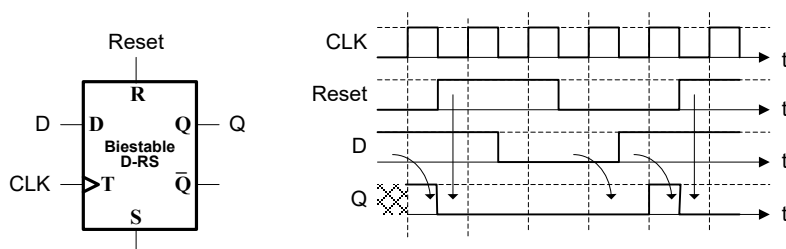


Figura 7.27. Estructura para implementar la función de *Reset* de manera asincrónica. Vemos que cuando esta señal está activada la salida se actualiza a cero inmediatamente mientras que en caso contrario se actualiza según la entrada *D* cuando llega un flanco de subida de la señal de reloj.

7.5.4. Implementación de la función *Set*

La señal externa de *Set* se usa para poner a uno los biestables. Como en el caso del *Reset* esto se puede realizar tanto de forma síncrona como asincrónica y las soluciones son prácticamente iguales: en vez de poner un cero en el multiplexor se pone un uno para el caso del funcionamiento síncrono, mientras que en vez de actuar sobre la señal asincrónica del biestable de tipo *Reset* se actúa sobre la *Set* para el segundo caso.

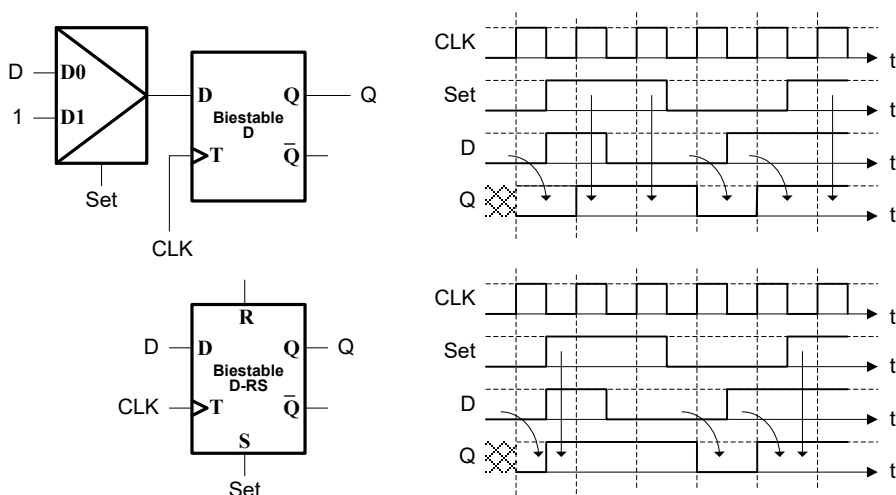


Figura 7.28. Estructuras para implementar la función de *Set* de manera síncrona (figura superior) o bien asincrónica (figura inferior). Vemos que cuando esta señal está activada la salida se actualiza a uno cuando llega un flanco de subida en el primer caso o bien inmediatamente en el segundo.

7.5.5. Prioridades en las señales de control

Una vez conocidas las estructuras básicas de los registros, así como la manera de implementar las funciones adicionales más habituales, sólo nos queda comentar cómo se controla la prioridad de estas señales en el supuesto que se active más de una de ellas en un momento dado.

Lo cierto es que no se puede explicar un método genérico, y explicar todas las combinaciones posibles no tiene mucho sentido, por lo que comentaremos los dos

puntos más importantes a tener en consideración y veremos distintos ejemplos que permitan extrapolar cualquier posible implementación.

- Cuando los biestables disponen de señales que actúan sobre sus señales asíncronas de *Reset* o bien de *Set* entonces cualquier priorización se ha de realizar implementado un sistema combinacional que aplicado a estas señales controle las priorizaciones que se establezcan.
- Cuando se han de controlar priorizaciones sobre señales que actúan sobre la entrada síncrona del biestable (*D*) entonces se suele realizar mediante distintos multiplexores que implementan las distintas señales de control siendo más prioritarias aquellas que queden más cercanas al biestable.

Ejemplo 7.7

Nos piden implementar un registro entrada serie – salida serie de 3 bits que tenga las siguientes características.

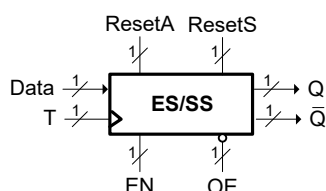


Figura 7.29. Registro ES/SS con dos posibles entradas de *Reset*, una con funcionamiento asíncrono y otra con funcionamiento síncrono. Además, dispone de una señal de *Enable* que es prioritaria sobre los dos *Reset*, así como un *Output Enable* activo por lógica negativa.

- *Data* es un canal de 1 bit por el que se introducen los datos en serie
- *T* es la señal de sincronismo del registro
- *OE* es una señal de *Output Enable* que funciona por lógica negativa
- *ResetA* es una señal de *Reset* que funciona de manera asíncrona
- *ResetS* es una señal de *Reset* que funciona de manera síncrona y es más prioritario que *ResetA*
- *EN* es una señal de *Enable* que es prioritario sobre los dos *Reset* del registro
- *Q* es un canal de 1 bit por el que salen los datos en serie
- $\bar{Q}$ es un canal de 1 bit por el que salen los datos invertidos en serie

La estructura básica serán tres biestables dispuestos en serie con un *tri-state* en el último biestable que activado por *OE* envíe los datos serie hacia la salida *Q* del registro. Haremos lo mismo para la señal $\bar{Q}$.

La señal *EN* la implementaremos tal y como hemos visto anteriormente mediante un multiplexor que aplicamos a la entrada de los biestables. Este multiplexor permitirá elegir entre la salida del biestable anterior o bien el mismo valor que cada biestable ya tuviese almacenado.

La señal *ResetS* también la implementaremos mediante un multiplexor que envíe un cero hacia la entrada del biestable. En cualquier caso, al ser menos prioritaria que *EN*, el multiplexor de *ResetS* lo ubicaremos más alejado de la entrada de cada biestable.

La señal *ResetA* actuará sobre la entrada de *Reset* asíncrona de los biestables. No obstante, sólo se activará siempre y cuando *EN* esté activa y *ResetS* esté desactivada al ser éstas dos más prioritarias que *ResetA*. Esto lo podemos implementar mediante una puerta AND de 3 entradas que sólo activará el *Reset* del biestable cuando se cumplan estas condiciones.

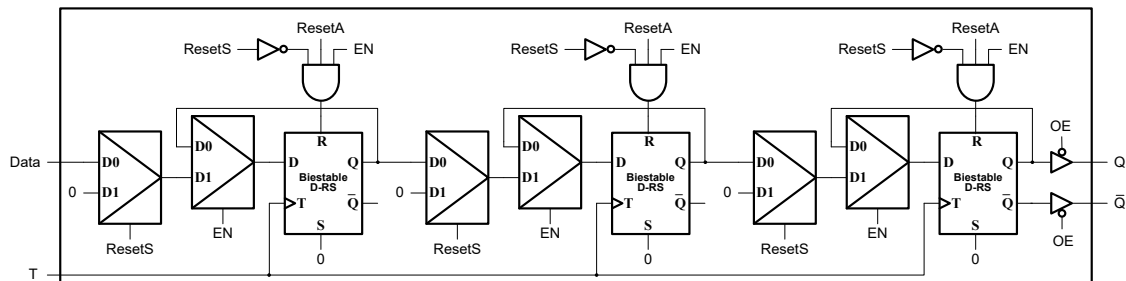


Figura 7.30. Implementación de un registro ES/SS con dos posibles entradas de *Reset*, una con funcionamiento asíncrono y otra con funcionamiento síncrono. Además, dispone de *Enable* que es prioritario sobre los dos *Reset*, así como un *Output Enable* activo por lógica negativa.

Ejemplo 7.8

Ahora nos piden implementar un registro entrada serie – salida paralelo de 3 bits que tenga las siguientes características.

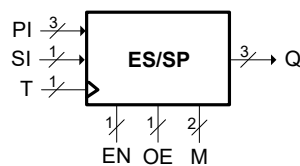


Figura 7.31. Registro ES/SP con entrada de *Enable* y *Output Enable*, así como una entrada M de dos bits que permite elegir entre distintos modos de funcionamiento.

- *PI* es un bus de 3 bits por el que se introducen los datos en paralelo
- *SI* es un canal de 1 bit por el que se introducen los datos en serie
- *T* es la señal de sincronismo del registro
- *EN* es una señal de *Enable* que es prioritaria sobre las otras señales
- *OE* es una señal de *Output Enable*
- *M* es un bus de 2 bits que indica el modo de funcionamiento
 - Cuando vale 00 se ha de efectuar un *Reset* asíncrono
 - Cuando vale 01 se ha de efectuar una carga síncrona en paralelo del valor de *PI*
 - Cuando vale 10 se ha de efectuar un desplazamiento serie desde *SI*
 - Cuando vale 11 se ha de efectuar un *Set* síncrono
- *Q* es un bus de 3 bits por el que salen los datos en paralelo

En este caso tendremos nuevamente una estructura serie con 3 biestables si bien ahora todas las salidas de los biestables salen hacia el exterior del registro y no sólo la correspondiente al último biestable como sucede en una salida serie.

La implementación de EN será como en el ejercicio anterior. Además, nuevamente es la más prioritaria, por lo que el correspondiente multiplexor estará lo más cercano posible a la entrada de datos de cada biestable.

La implementación de M la efectuaremos mediante otro multiplexor que elegirá entre una u otra opción. Cuando sea 00 el valor será indiferente (aunque pondremos un cero) puesto que el *Reset* se efectuará sobre la entrada asíncrona del biestable. Cuando sea 01 el valor que el multiplexor enviará al biestable será el correspondiente al bus exterior de 3 bits. Cuando sea 10 el valor que el multiplexor enviará al biestable será el que envíe el biestable anterior. Cuando sea 11 el valor que el multiplexor enviará al biestable será un 1 puesto que hemos de efectuar un *Set* síncrono.

La entrada asíncrona de *Reset* del biestable se activará cuando M valga 00 y además esté habilitada la señal EN al ser ésta más prioritaria.

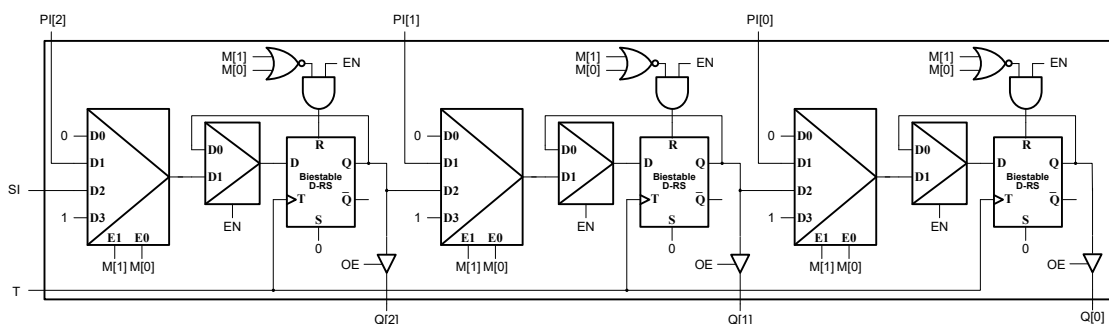


Figura 7.32. Implementación de un registro ES/SP con entrada de *Enable* y *Output Enable*, así como una entrada M de dos bits que permite elegir entre distintos modos de funcionamiento.

Vista esta solución os proponemos que os planteéis cómo habría variado la solución si la carga en paralelo del valor de PI se hubiese de efectuar de manera asíncrona en vez de síncrona. Lógicamente en este caso tendremos que actuar sobre las entradas asíncronas de los biestables para que, según el valor del bit correspondiente de PI , actuemos sobre el *Reset* si éste vale cero o bien sobre el *Set* si éste vale uno. Por consiguiente, tenemos que plantearnos sistemas combinacionales en las entradas de *Reset* y *Set* de los biestables para controlarlo. Os proponemos que intentéis hallar una posible solución.

Ejemplo 7.9

Para practicar otro ejemplo os proponemos que implementéis un registro entrada paralelo – salida serie de 4 bits con las siguientes entradas:

- $Data$ será un bus de 4 bits por donde se introducirán los valores de entrada
- T es la señal de sincronismo del registro
- $L/\bar{S}$ es una señal que elige el modo de funcionamiento:
 - Si vale 1 se efectúa una carga paralelo de los datos de entrada
 - Si vale 0 se efectúa un desplazamiento en serie
- R es una señal de *Reset* síncrona y es más prioritaria que $L/\bar{S}$
- S es una señal de *Set* asíncrona y es más prioritaria que R
- EN es una señal de *Enable* que es prioritaria sobre las otras señales
- Q es un canal de 1 bit por el que salen los datos en serie

7.6. Problemas de diseño con registros

Tal y como hemos visto, los registros son elementos que permiten almacenar un conjunto de bits para posteriormente utilizarlos según precisemos. Dicho almacenamiento y recuperación de información puede realizarse en paralelo o en serie, dando lugar a las cuatro tipologías básicas de registros: EP/SP, ES/SS, ES/SP y EP/SS. Además, los registros pueden disponer de señales de control adicionales tipo *Enable*, *Output Enable*, *Reset*, etc. con las que podremos controlar funciones adicionales.

Una vez comprendido el funcionamiento de los registros y de haber visto cómo es su estructura interna a nivel de biestables, bloques funcionales y puertas lógicas, en este apartado os proponemos ver su uso en el momento de realizar el diseño de ciertas aplicaciones y poder apreciar en qué circunstancias puede tener sentido utilizar registros y cómo dimensionarlos y unirlos al resto de elementos del circuito.

No existe una sistemática a seguir en su utilización en problemas de diseño y sólo la experiencia acaba facilitando el bagaje necesario para saber cuándo y cómo se han de utilizar. Antes de ver los ejemplos comentar que éstos tienen un carácter básicamente docente y por consiguiente han estado preparados para ayudar a comprender el uso de registros ante posibles requerimientos. Las aplicaciones reales suelen incluir conceptos con los que aún no hemos trabajado.

Ejemplo 7.10

Nos piden diseñar un sistema que realice unos cálculos concretos. El sistema en cuestión dispone de las siguientes entradas y salidas.



Figura 7.33. Sistema a diseñar

- *Op*: bus de 2 bits que indicará el tipo de cálculo a realizar.
- *DataIn*: bus de 8 bits por el que el usuario introduce los números con los que se han de realizar los cálculos. Los números se introducen en complemento a 2. En cualquier caso, el sistema tendrá que tener almacenados siempre los tres últimos números introducidos puesto que éstos pueden ser los que se requieran para efectuar los cálculos.
- *Enter*: señal de 1 bit que el usuario activará por lógica positiva para indicar que en *DataIn* hay un nuevo número a introducir en el sistema.
- *Send*: señal de 1 bit que el usuario activará por lógica positiva para indicar que el sistema facilite un único bit del resultado del cálculo a realizar por la línea *DataOut*. Se trata por consiguiente de un sistema que transmite el resultado del cálculo en serie, facilitando dicho resultado bit a bit, a medida que el usuario activa *Send*.
- *DataOut*: línea de salida de 1 bit por la que el sistema extrae en serie el resultado de la operación.

- Tx : línea de salida de 1 bit que indicará por lógica positiva que por *DataOut* se está extrayendo el resultado de una operación. De esta manera el usuario sabrá que cuando Tx valga cero el sistema habrá finalizado el envío del resultado y entonces puede solicitar un nuevo cálculo.

El sistema tal y como se ha comentado realiza un cálculo a partir de los tres últimos números introducidos y dependiendo del valor de Op .

- Si Op es 00 el sistema ha de calcular el mayor de los últimos tres números introducidos (Max)
- Si Op es 01 el sistema ha de calcular el menor de los últimos tres números introducidos (Min)
- Si Op es 10 el sistema ha de calcular la media del mayor y el menor de los últimos tres números introducidos $\frac{1}{2}(\text{Max}+\text{Min})$
- Si Op es 11 el sistema ha de calcular la diferencia entre el mayor y el menor de los últimos tres números introducidos (Max-Min)

Para comprender mejor el funcionamiento supongamos un caso. Imaginemos que primero el usuario introduce el +56 en *DataIn* (lógicamente en complemento a 2), pulsa *Enter*, luego introduce el -14 en *DataIn*, pulsa *Enter* y luego el +37 y pulsa *Enter*. Imaginemos ahora que cuando pulsa *Send* tenemos que Op es 10, entonces por *DataOut* saldrá un bit del resultado del cálculo $\frac{1}{2}(+56-14)$. En este mismo momento activará Tx a 1 para indicar que por *DataOut* está saliendo información. A medida que el usuario vaya activando *Send* irán saliendo los bits sucesivos del anterior cálculo hasta que una vez obtenido el último bit el sistema ponga Tx a 0 para informar al usuario que ya no hay más bits de resultado. Si ahora el usuario pone -127 en *DataIn* y pulsa *Enter*, entonces el sistema realizará el nuevo cálculo utilizando como posibles datos los números -127, +37 y -14 al ser los tres últimos valores introducidos. De este modo si cuando pulsa *Send* tenemos que Op es 00 entonces el sistema sacará el primer bit del resultado de la nueva operación, que es obtener el máximo, es decir el primer bit del +37. Y así sucesivamente. Supondremos también que el usuario es buena persona y mientras Tx valga uno no introducirá ningún número nuevo ni cambiará el valor de Op hasta que hayan salido todos los bits de la última operación.

Veamos una posible solución, puesto que estos problemas de diseño habitualmente pueden resolverse de múltiples maneras.

Vemos que todas las operaciones han de realizarse considerando los tres últimos números introducidos por lo que utilizaremos tres registros EP/SP dispuestos en serie que compartirán la misma señal de sincronismo (*Enter*) de tal manera que en todo momento en estos tres registros dispondremos los tres últimos números introducidos. Así cada vez que el usuario pulse *Enter* desplazaremos el último y penúltimo número introducido “hacia delante” e introduciremos en el último registro el nuevo número. A estos tres números los denominaremos N_1 , N_2 y N_3 puesto que los utilizaremos en el resto de bloques.

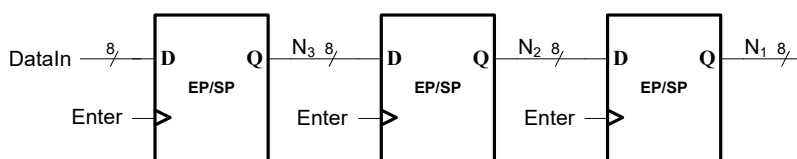


Figura 7.34. Almacenamiento de los datos de entrada. Cada vez que el usuario pulsa *Enter* introduciremos un nuevo dato en el primer registro (de la izquierda) y simultáneamente desplazamos el último y penúltimo valor introducido hacia los otros dos registros.

Diseñaremos ahora un módulo que nos permita determinar cuál de los tres números es el mayor y cuál es el menor, puesto que estos valores los necesitaremos para realizar las distintas operaciones. Una posible solución podría pasar por utilizar comparadores, pero otra opción puede pasar por restar los números de dos en dos y según el resultado, positivo o negativo, sabremos cuál de los dos números es mayor. La opción de utilizar un comparador o bien un restador es muy similar y la primera es más intuitiva. No obstante, hemos de tener en consideración que los comparadores sólo permiten comparar números en binario natural y si decidimos optar por utilizar comparadores tendremos que añadir elementos que controlen la posible conversión entre formatos numéricos. Por consiguiente, elegiremos la opción de los restadores.

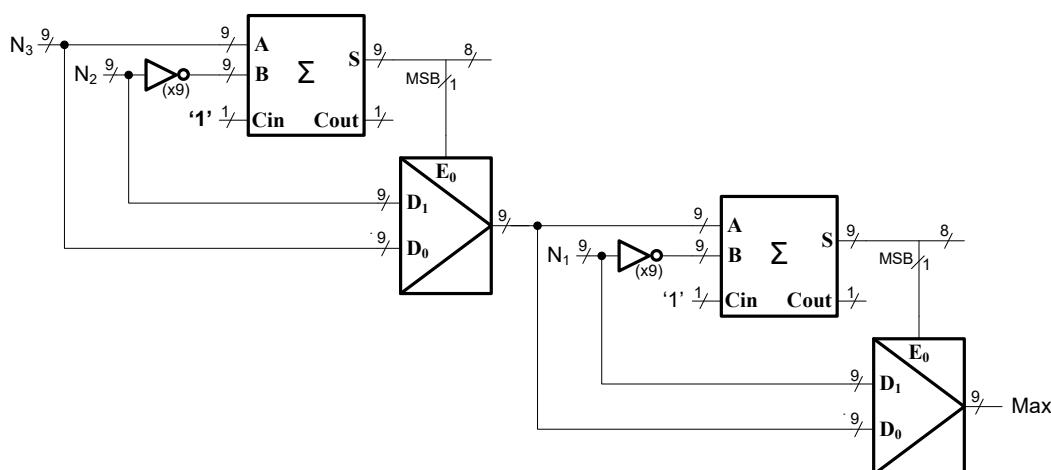


Figura 7.35. Obtención del valor máximo. Restamos N_3 y N_2 de tal manera que según el signo del resultado sabremos cuál de los dos es mayor, valor que es elegido mediante un multiplexor.

Posteriormente restamos este último valor con N_1 y nuevamente según el signo de la resta sabremos y elegiremos cuál es el valor máximo. Podemos observar que los números N_1 , N_2 y N_3 los hemos extendido a 9 bits (replicando el bit de más peso) puesto que en caso contrario la resta podría dar un resultado incorrecto.

De manera similar podemos buscar cuál es el menor de los tres números almacenados. Utilizaríamos la misma solución sólo que intercambiando las entradas de los multiplexores.

A continuación, mostraremos cuál hubiese sido la solución en el supuesto de haber utilizado comparadores para poder ver una alternativa a la solución anterior. En cualquier caso, debe quedar claro que esta solución no sería válida para este problema pues sólo funciona si los números están codificados en binario natural y no en complemento a dos como es nuestro caso.

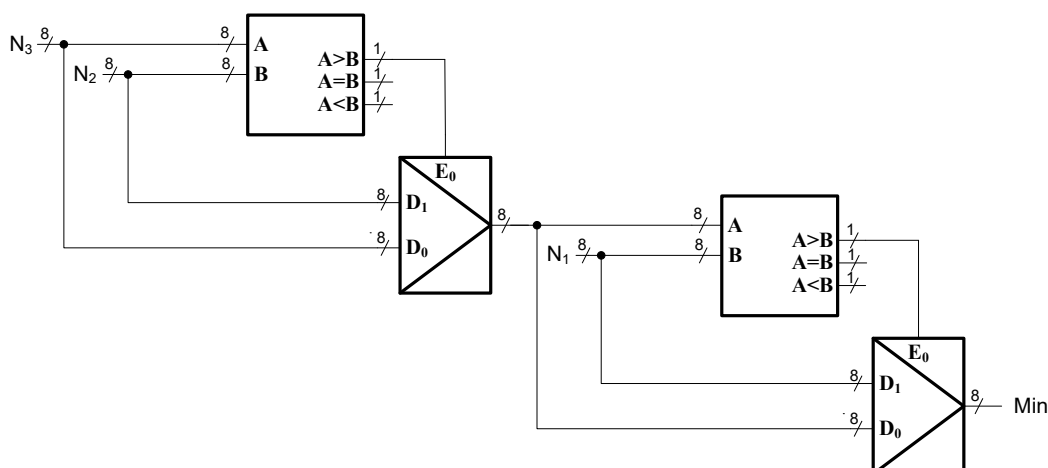


Figura 7.36. Posible método de obtención del valor mínimo mediante comparadores. Comparamos N_3 y N_2 de tal manera que según el resultado sabremos cuál de los dos es mayor, valor que es elegido mediante un multiplexor. Posteriormente comparamos este último valor con N_1 y nuevamente según el resultado sabremos y elegiremos cuál es el valor mínimo. Esta solución no sería aplicable al problema propuesto puesto que sólo funciona en binario natural y no en complemento a dos.

Con los valores máximos y mínimos de los tres últimos números introducidos disponemos ya de los resultados correspondientes a las operaciones que han de realizarse cuando Op sea 00 o bien 01. Ahora diseñaremos, mediante un sumador, un bloque sumador – restador que nos permita realizar el cálculo tanto para cuando Op sea 10 como cuando sea 11. Para el primer caso tendremos que calcular $\frac{1}{2}(\text{Max} + \text{Min})$ mientras que para el segundo caso tendremos que calcular $(\text{Max} - \text{Min})$. Este bloque por consiguiente puede utilizar el bit de menor peso de Op para saber si ha de realizar una suma o bien una resta. En este último caso tendremos que invertir el valor a restar y sumar la unidad al sumador, tal y como corresponde a una resta en complemento a dos. La división por 2 para cuando Op sea 10 la implementaremos a la salida del sumador extrayendo el bit de menor peso del resultado.

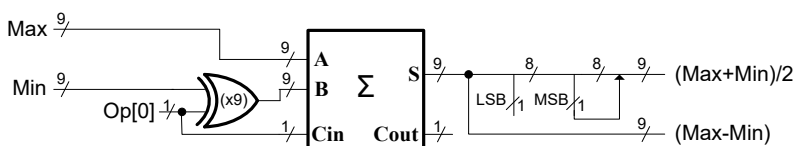


Figura 7.37. Bloque sumador – restador que ha de permitir calcular las operaciones correspondientes de suma y resta necesarias. Primero invertimos, mediante una puerta XOR (de hecho, son 9 puertas, una por cada bit) y sumamos la unidad cuando tengamos que realizar una resta. Finalmente, a la salida realizamos una división por dos para cuando Op sea 10.

Finalmente, mediante un multiplexor podremos elegir cuál ha de ser el resultado que tendremos que enviar en serie según la entrada Op . Vemos que las líneas de entrada y salida son de 9 bits puesto que ya hemos visto que realizamos operaciones en las que el resultado puede precisar de esta longitud para codificarse correctamente.

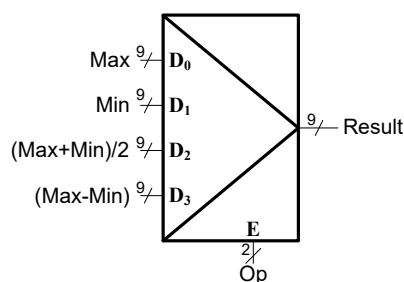


Figura 7.38. Multiplexor que según Op determinará el valor que tendremos que facilitar como resultado.

Tan solo nos queda enviar este resultado en serie a medida que el usuario active *Send*. Para ello almacenaremos el resultado en un registro EP/SS cuando Tx valga cero y el usuario pulse *Send* para posteriormente ir enviando bit a bit a medida que continúe pulsando *Send*. La señal Tx se ha de poner a 1 cuando se active *Send* y se ha de mantener en este valor hasta que se hayan enviado todos los 9 bits del resultado. Para controlar este punto podemos crear un bloque contador que nos avisará cuando hayamos enviado todos los bits y por consiguiente volvamos a poner a cero el contador y estemos en disposición de volver a cargar un nuevo resultado cuando el usuario vuelva a pulsar *Send*. Dicho contador se puede implementar mediante un registro que a cada flanco almacena el resultado de sumar el valor que ya tenía anteriormente más uno.

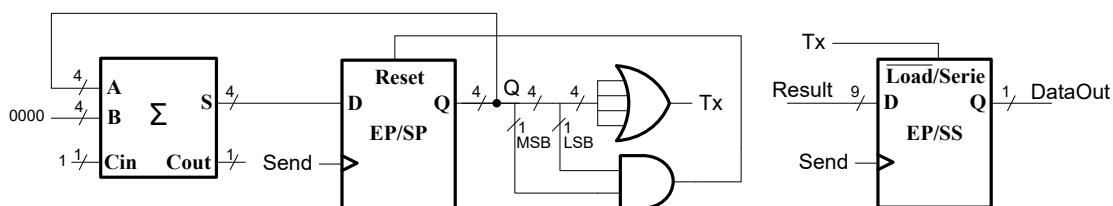


Figura 7.39. Contador de 4 bits implementado con un sumador y un registro EP/SP que utilizaremos para contar cuántos bits hemos enviado hasta que llegado a la combinación 1001, es decir 9 en decimal, volvamos a poner el contador a cero para cuando tengamos que enviar el siguiente resultado. Mediante este contador generamos la señal Tx que valdrá uno mientras estemos contando hasta 9 la cual controlará si en el registro de salida EP/SS tenemos que efectuar una carga del siguiente valor a enviar o bien enviamos los bits en serie.

Para comprender mejor el funcionamiento de este último bloque supongamos el siguiente cronograma en el que vemos como a medida que se genera una señal de *Send* se va incrementando el valor del contador y además se va transmitiendo en serie el resultado de la operación hasta que llegado el último bit, se genera entonces un *Reset* i el sistema queda a la espera como en el estado inicial, a la espera de enviar un nuevo resultado.

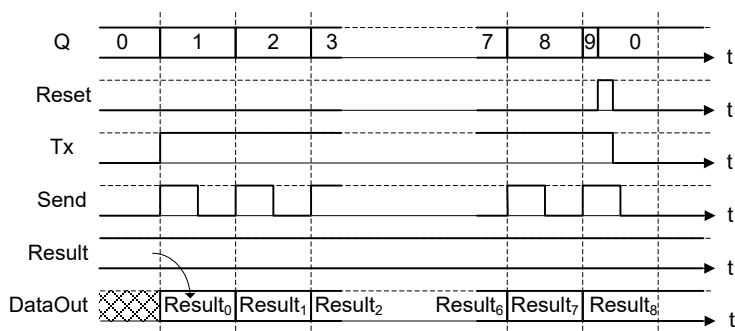


Figura 7.40. Cronograma sobre el funcionamiento del contador de 4 bits. Supongamos que inicialmente el contador está a cero (fila superior) y en el bus *Result* tenemos la información que queremos enviar. El usuario entonces dará una consigna de *Send* que, además de incrementar el contador y ponerlo a 1, cargará el registro EP/SS con el resultado y dicho registro de salida visualizará el primer bit del valor que hemos cargado. Posteriormente a cada nuevo flanco de *Send* el contador se incrementará y se enviará un nuevo bit del resultado que inicialmente fue cargado. Finalmente, el proceso de envío finalizará cuando, generado un flanco que ponga el contador a 9, puesto que, además de poner el último bit del resultado, activará la señal de *Reset* del contador y desactivará *Tx* dejando el bloque a la espera de un nuevo proceso de envío de información.

Todos estos bloques que hemos diseñado por etapas constituyen, en su conjunto, una posible arquitectura que implementa una solución al problema planteado.

Ejemplo 7.11

Nos piden diseñar un sistema que a partir de unos parámetros que nosotros mismos fijemos correspondientes a un viaje, nos calcule si lo podemos realizar o no según un presupuesto que tengamos asignado. De hecho, nuestro viaje incluye una parte cultural y otra de descanso, de manera que tendremos dos tipos de hoteles: los normales para la parte cultural y los resorts para los días de descanso. Además, durante los días de descanso también puede ser que nos dediquemos a hacer submarinismo. El sistema a diseñar dispone de las siguientes entradas y salidas.

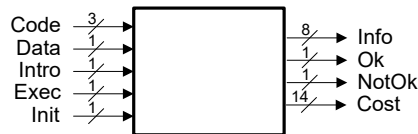


Figura 7.41. Sistema evaluador de la posibilidad de efectuar un viaje según el cálculo del coste del mismo y el presupuesto que disponemos.

- *Code*: bus de 3 bits por los que nos especifican el tipo de información que se introduce en el sistema por la línea *Data*. Los posibles valores de *Code* son los siguientes:
 - 000 para indicar el precio del vuelo (ida y vuelta) codificado con 11 bits
 - 001 para indicar el precio por noche del hotel normal codificado con 8 bits
 - 010 para indicar el tipo de resort que elegimos codificado en 1 bit. Si vale 0 el precio por noche del resort es el mismo precio que un hotel normal mientras que si vale 1 el precio por noche del resort es el doble del precio por noche del hotel normal.
 - 011 para indicar el número de inmersiones diarias codificadas con 3 bits
 - 100 para indicar el precio de cada inmersión codificado con 8 bits
 - 101 para indicar el número total de días codificados con 5 bits
 - 110 para indicar el número de días que estaremos en el resort codificados con 5 bits
 - 111 para indicar el máximo presupuesto que disponemos codificado con 14 bits
- *Data*: línea de 1 bit por la que se introduce el concepto especificado en *Code* al ritmo que especifica la señal *Intro*. Los valores se introducen en binario natural.
- *Intro*: línea de 1 bit por la que llegan los flancos correspondientes al número de bits que tenga el valor que se introduce. A cada flanco de *Intro* se introduce un bit de información por *Data*.
- *Init*: línea de 1 bit que se usa para inicializar el sistema poniendo a cero todos los posibles valores internos.

- *Info*: bus de 8 bits que nos indica con un 1 los conceptos que se han introducido según la tabla de valores de *Code*. Así, si en un momento dado hemos introducido el número de inmersiones diarias (*Code* vale 011, es decir, 3 en binario) y el número de días en el resort (*Code* vale 110, es decir, 6 en binario) entonces los bits 3 y 6 del bus *Info* han de valer 1 y por consiguiente *Info* será 01001000
- *Cost*: bus de 14 bits que muestra el coste del viaje.
- *Exec*: línea de 1 bit que indica que se ha de evaluar si el viaje se puede realizar o no según el presupuesto disponible.
- *Ok*: línea de 1 bit que indicará que el coste del viaje no excede el presupuesto máximo. Sólo se puede activar cuando se han introducido todos los datos y no hay ningún error de cálculo.
- *NoOk*: línea de 1 bit que indicará que el coste del viaje excede el presupuesto máximo o bien que se ha producido un error de cálculo. Sólo se puede activar cuando se han introducido todos los datos.

Nuevamente nos hallamos ante un problema de diseño que podemos mirar de realizar modularmente. Empezaremos almacenando la información que nos llegue por *Data* en registros ES/SP, ordenándola según el valor de *Code*. Para ello proponemos utilizar un decodificador que según *Code* habilitará uno u otro registro, el cual almacenará los bits a medida que lleguen flancos de *Intro*.

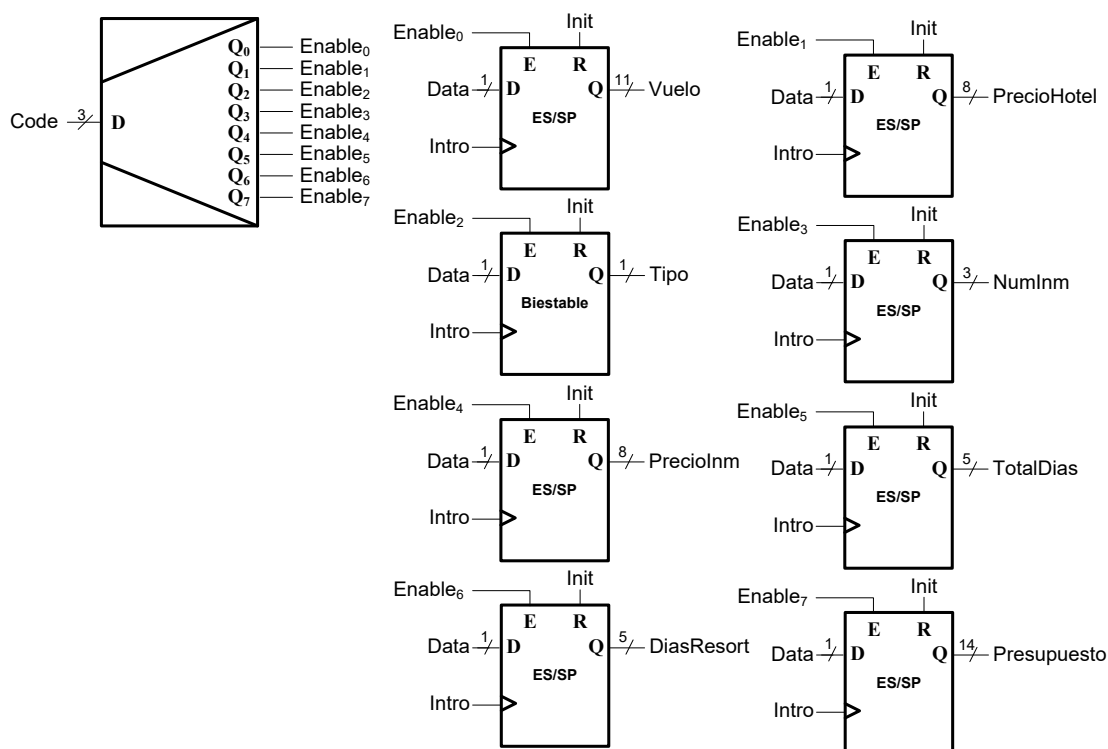


Figura 7.42. Almacenamiento de los datos. Según *Code* tendremos habilitado uno u otro registro. De esta manera a medida que vayamos recibiendo flancos de *Intro* se irá introduciendo en serie el dato en el registro correspondiente.

A medida que se van activando las salidas del anterior decodificador podemos ir activando el bit correspondiente del bus de salida *Info* que indica los datos que hemos recibido. Almacenaremos esta información en biestables de tipo R-S para que se mantenga hasta que se vuelva a inicializar el sistema mediante *Init*.

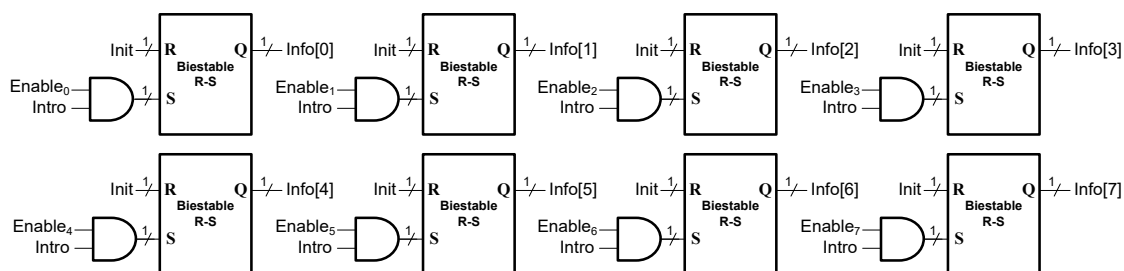


Figura 7.43. Activación de las líneas correspondientes al bus *Info* a medida que llegan los correspondientes datos.

Ahora que ya hemos diseñado el almacenamiento de los datos podemos proceder a diseñar el bloque que realice el cálculo del coste del viaje. Por un lado calcularemos el coste diario de las inmersiones multiplicando el número de inmersiones diarias por el precio de cada inmersión. A esto le sumaremos el precio del hotel de los días que estemos en el resort y tendremos el coste diario de los días de resort, por lo que multiplicado por los días de resort tendremos el coste total del resort. El precio del hotel del resort depende del valor de *Tipo* por lo que mediante un multiplexor elegiremos un cierto precio u otro. Por otro lado, restaremos, convirtiendo los números a complemento a 2, los días de resort de los días totales para saber los días que estaremos en un hotel normal y lo multiplicaremos por el precio de hotel. Finalmente quedará sumar el precio de los vuelos y el coste que habremos calculado del resort, con lo que ya tendremos el coste total del viaje.

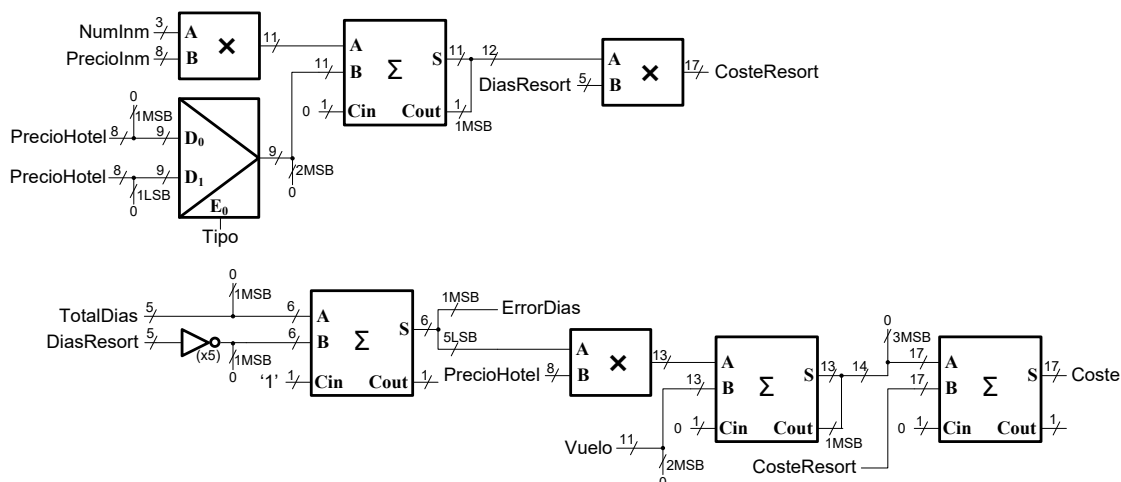


Figura 7.44. Cálculo del coste del viaje previsto en función de los diferentes parámetros. Adicionalmente vemos que generamos una señal de error en el supuesto que el usuario introduzca más días de resort que el número total de días del viaje.

El valor del coste previsto del viaje ya lo podemos sacar por el bus de salida denominado *Cost*. No obstante, como dicho bus tiene una longitud fijada a 14 bits sólo sacaremos los 14 bits de menor peso y, en caso que alguno de los tres bits de mayor peso fuese uno, sabríamos que con toda seguridad el coste sobrepasará el presupuesto disponible pues éste tiene una longitud también fijada en 14 bits. Además, podemos calcular mediante un comparador si sobrepasamos el presupuesto disponible.

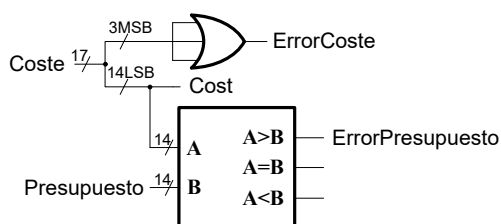


Figura 7.45. Determinación de si el coste del viaje es superior al posible presupuesto.

Finalmente, sólo queda controlar que cuando el usuario active *Exec* el sistema avise si se puede realizar o no el viaje. Además, y tal y como se comenta en el planteamiento del problema, sólo se permitirá la activación cuando podamos asegurar que todos los datos han sido introducidos.

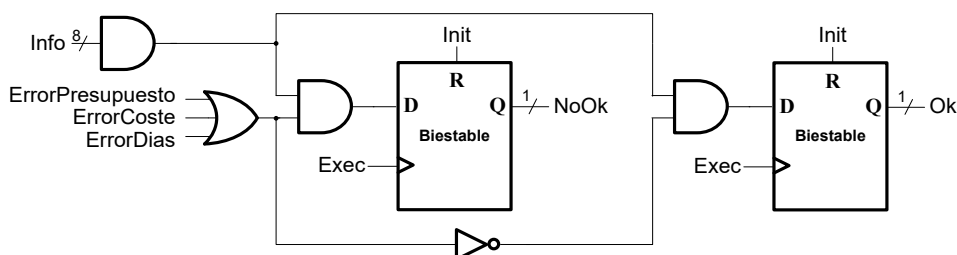


Figura 7.46. Generación de las señales de salida que indicarán si el viaje se puede realizar.

Ejemplo 7.12

Para practicar los problemas de diseño os proponemos que diseñéis un bloque de cálculo estadístico para calcular tres operaciones básicas: la suma de los datos de entrada, la suma de los valores absolutos de los datos y el valor absoluto de la suma de los datos de entrada. El diagrama del bloque es el siguiente:

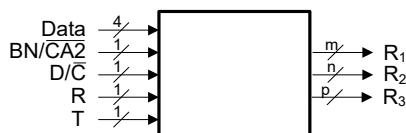


Figura 7.47. Sistema de cálculo estadístico.

- *T*: sincronismo del sistema activo por flanco de subida
- *R*: reset asíncrono del sistema que ha de poner todos los valores a cero
- *D/C*: línea que indica con un 1 que cuando llega un sincronismo se tiene que cargar un nuevo valor al sistema o con un 0 que cuando llega un sincronismo se tiene que facilitar a la salida el resultado de las tres operaciones previstas
- *Data*: bus de entrada de 4 bits por el que el usuario introduce los valores con los que se han de realizar las operaciones, de tal modo que cada vez que se activa un sincronismo y la señal *D/C* vale 1 el sistema tomará el valor que esté en este bus para tenerlo en consideración en el momento de efectuar los posibles cálculos
- *BN/CA2*: línea de 1 bit que indicará con un 1 si el valor que en ese momento está en *Data* está codificado en binario natural o con un 0 si lo está en complemento a 2.

El funcionamiento del sistema es relativamente simple. Supongamos el sistema que parte de todos los posibles valores internos a cero (consecuencia de un *Reset*). El usuario irá introduciendo valores al sistema por el bus *Data*, con $D/\bar{C}$ a 1 y activando para cada valor que quiera introducir un sincronismo. Una vez haya introducido todos los valores y quiera obtener el resultado entonces pondrá $D/\bar{C}$ a 0 y generará un nuevo sincronismo. En ese instante el sistema ha de mostrar por los tres buses de salida el resultado en complemento a 2 de 1) la suma de todos los valores introducidos (por el bus R_1), 2) la suma de los valores absolutos de todos los valores introducidos (por el bus R_2) y 3) el valor absoluto de la suma de todos los valores introducidos (por el bus R_3). Finalmente, y antes de generar un nuevo cálculo se supone que el usuario volverá a activar el *Reset* para partir nuevamente de un estado inicial sin valores almacenados.

A partir de este enunciado os proponemos que planteéis alguna posible solución al problema, diseñando los distintos bloques que consideréis necesarios. Para poder dimensionar la solución consideraremos que el número máximo de valores que el usuario puede introducir para efectuar los cálculos es de 10.

Una vez planteada la solución podéis plantearos también algún mecanismo para enviar los valores de salida en serie.

7.7. Registros en VHDL

Recordemos que la implementación de elementos de memorización en VHDL se realiza no especificando el valor que ha de adoptar la salida para todas las posibles situaciones de entrada. En ese caso, el software interpreta que queremos implementar un elemento con memoria, y hace que para cualquier combinación de las entradas no contemplada dentro de las diferentes partes de la descripción del comportamiento de esa entidad se mantenga el último valor conocido de las salidas. La descripción de estos elementos de memorización se suele realizar mediante el uso de sentencias *if* dentro de procesos. Veamos algunos ejemplos de descripción de registros en VHDL.

7.7.1. Registro EP/SP en VHDL

La descripción de un registro EP/SP básico (sin enable, reset...) podría ser la siguiente:

```
library ieee;
use ieee.std_logic_1164.all;

entity Reg_bas_8bit is
    port (
        Clk      : in      std_logic;
        D        : in      std_logic_vector(7 downto 0);
        Q        : out     std_logic_vector(7 downto 0);
    );
end Reg_bas_8bit;
```

architecture behavioural of Reg_bas_8bit is

begin

process (Clk) is

begin

if (Clk'event and Clk = '1') then

Q <= D;

end if;

end process;

end behavioural;

El resultado de la simulación de este registro nos da el siguiente diagrama de tiempos:

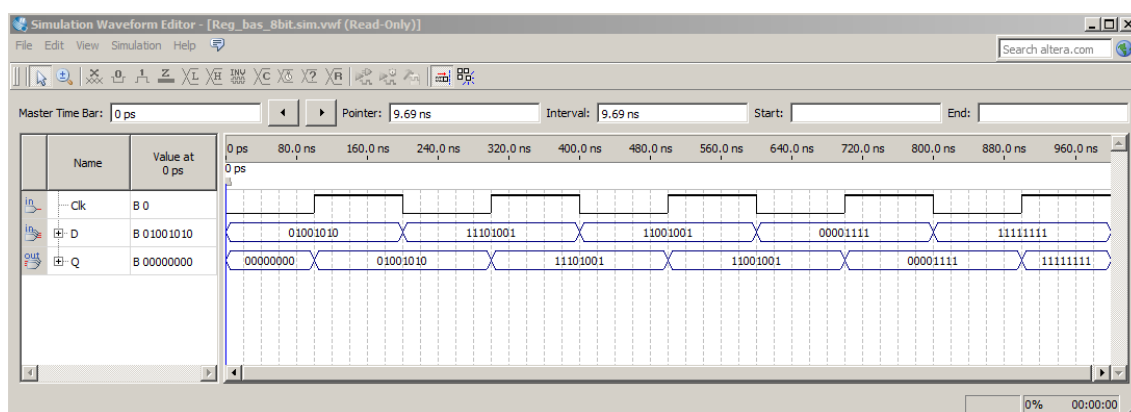


Figura 7.48. Simulación del registro EP/SP de 8 bits descrito según el código VHDL propuesto.

Como se puede observar, la salida del registro se actualiza con el valor presente en su entrada en cada flanco de subida de la señal de sincronismo. El valor inicial (todo ceros) de la salida está forzado por la inicialización por defecto de las señales que hace el software.

Dejamos para el lector la descripción en VHDL de un registro EP/SP de 4 bits con, por ejemplo, señales de control de enable y de reset asíncrono.

7.7.2. Registros de desplazamiento en VHDL

En VHDL existe el operador de concatenación que permite realizar desplazamientos (*shift*) de los bits que conforman una cierta combinación. Este operador se representa en VHDL por el símbolo "&".

Así, si por ejemplo tenemos un vector de 4 bits llamado A, es decir, una señal A de tipo `std_logic_vector(3 downto 0)`, los 4 bits individuales automáticamente pasan a denominarse A(3) (el de más peso), A(2), A(1) y A(0) (el de menos peso). De esta forma,

podemos construir una señal B de 4 bits concatenando los bits de A en el orden que queramos. Por ejemplo, para invertir el peso de los bits (que los de más peso pasen a ser los de menos peso y viceversa), podemos hacer la siguiente asignación:

```
B <= A(0) & A(1) & A(2) & A(3);
```

obteniéndose de esta manera $B(3) = A(0)$, $B(2) = A(1)$, $B(1) = A(2)$ y $B(0) = A(3)$.

Introduciremos también una forma de describir el comportamiento de los registros que pueden memorizar una u otra información mediante la combinación de un registro EP/SP “simple” y un sistema combinacional (normalmente un multiplexor) en su entrada que calcula el siguiente valor a memorizar en el registro. Denominaremos Q_reg a la salida del registro y Q_next a su entrada, que pasará a ser la nueva información memorizada en el registro con el siguiente flanco de la señal de sincronismo. Así, el funcionamiento del registro EP/SP se realizará mediante su propio proceso, sensible a la señal de reloj y, si las hubiere, a las señales de enable o reset:

```
process (Clk) is
begin
    if (Clk'event and Clk = '1') then
        Q_reg <= Q_next;
    end if;
end process;
```

Ahora basta con añadir un sistema combinacional que “calcula” Q_next a partir de las entradas y/o de la propia información memorizada en el registro, Q_reg. Si por ejemplo queremos hacer un “*shift right*” (desplazar los bits una posición hacia la zona de menos peso introduciendo un cero como bit de más peso) lo podríamos hacer de la forma siguiente:

```
Q_next <= '0' & Q_reg(3 downto 1);
```

Finalmente, si queremos describir un registro de 4 bits, con reset asíncrono (lógica positiva) y con una señal de control “Load” tal que si vale “0” el registro realiza un “*shift right*” y si vale “1” realiza una carga en paralelo de los 4 bits presentes en su entrada D lo podríamos hacer de esta manera:

```
library ieee;
use ieee.std_logic_1164.all;

entity Reg_shift_right is
    port
    (
        Clk, Load, Reset : in    std_logic;
        D                 : in    std_logic_vector(3 downto 0);
        Q                 : out   std_logic_vector(3 downto 0));
end Reg_shift_right;

architecture behavioural of Reg_shift_right is

    signal Q_Reg, Q_Next : std_logic_vector(3 downto 0);
```

```

begin

Q_Next <= D when Load = '1' else '0' & Q_Reg(3 downto 1);

process (Clk, Reset) is
begin
    if Reset = '1' then
        Q_reg <= (others => '0');
    elsif (Clk'event and Clk = '1') then
        Q_Reg <= Q_Next;
    end if;
end process;

Q <= Q_reg;

end behavioural;

```

Observemos cómo ha sintetizado el Quartus Prime nuestro diseño, combinando un registro EP/SP con un multiplexor:

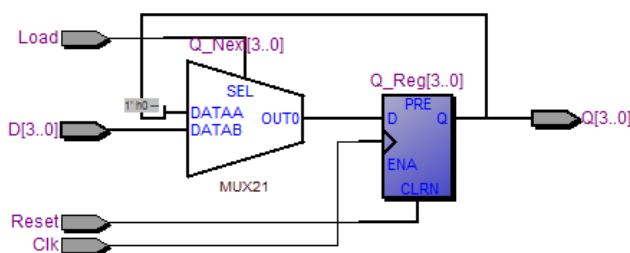


Figura 7.49. Síntesis del registro con “*shift right*” realizada por el Quartus Prime.

El resultado de la simulación de este registro nos da el siguiente diagrama de tiempos:

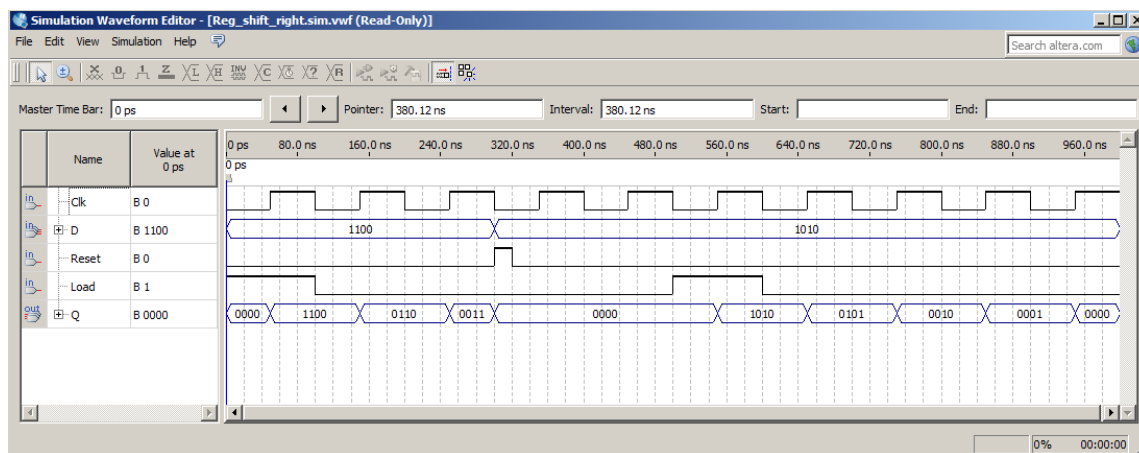


Figura 7.50. Simulación del registro descrito según el código VHDL propuesto.